



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Prueba Practica – Tower Defense
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3 – “B”
Alumnos participantes:	Curillo Pilamunga Diego Alexander Manobanda Masabanda Luis Alfredo Quispe Nasimba Daniela Alejandra Redroban Chimbo Joselyn Amparo
Asignatura:	Estructura de datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar un juego Tower Defense en C++ que implemente estructuras de datos lineales dinámicas de forma manual, permitiendo al jugador construir torres defensivas en posiciones estratégicas para detener oleadas de enemigos que avanzan por un camino lineal, gestionando recursos económicos y tomando decisiones tácticas para proteger su base hasta completar todas las oleadas o hasta que sus vidas se agoten.

Específicos:

- Diseñar e implementar tres estructuras de datos fundamentales sin utilizar las bibliotecas estándar de contenedores de C++ (vector, list, queue, deque), desarrollando manualmente una lista doblemente enlazada para gestionar los enemigos activos en el campo de batalla, una lista secuencial dinámica para administrar las torres construidas por el jugador, y una lista circular para organizar las oleadas de enemigos.
- Desarrollar el sistema de combate que procese cada turno siguiendo un orden específico donde las torres atacan antes de que los enemigos se muevan, evitando que enemigos con alta velocidad puedan atravesar el rango de las torres sin recibir daño, asegurando que las defensas tengan oportunidad de impactar a los enemigos antes de que avancen hacia la base del jugador.
- Diseñar un sistema económico y de progresión que inicie con \$400 de dinero inicial, donde cada torre tenga un costo específico según su tipo, los enemigos destruidos otorguen recompensas variables según su dificultad, y se implemente un sistema de 3 vidas donde cada vida tenga 100 puntos de salud, permitiendo al jugador tomar decisiones estratégicas y sentir el avance a través del juego.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 3

No presenciales: 3



2.4 Instrucciones

Detalladas en la guía proporcionada por el docente.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- **Laltop, dev,**

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☒ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Desarrollo

Introducción

El presente documento detalla la implementación técnica del juego Tower Defense desarrollado en C++. El proyecto implementa manualmente todas las estructuras de datos requeridas sin utilizar las bibliotecas estándar de contenedores como vector, list, queue o deque. Se utilizan únicamente las bibliotecas básicas iostream, string, iomanip, algorithm y limits para garantizar el cumplimiento de las restricciones establecidas.

El juego simula un campo de batalla lineal con posiciones que van desde 0 hasta 20. El jugador debe construir torres defensivas en diferentes posiciones para detener a las oleadas de enemigos que avanzan desde la posición 0 hasta la posición 20. Cada enemigo que alcanza la posición final reduce la vida del jugador, mientras que los enemigos destruidos otorgan recompensas en dinero que permiten construir más torres.

Estructuras de Datos

NodoEnemigo

La estructura NodoEnemigo representa cada enemigo individual en el campo de batalla. Contiene toda la información necesaria para el comportamiento del enemigo durante el combate. El campo id es un identificador único que se genera automáticamente al crear cada enemigo, permitiendo su identificación y búsqueda posterior. El campo tipo almacena la categoría del enemigo, que puede ser "normal", "rapido" o "fuerte", determinando sus estadísticas base. La vida representa los puntos de salud actuales del enemigo, que disminuyen cuando recibe ataques de las torres. La velocidad determina cuántas posiciones avanza el enemigo en cada turno. La posicion indica su ubicación actual en el camino lineal que va de 0 a 20. La recompensa es la cantidad de dinero que recibe el jugador cuando el enemigo es destruido. Los punteros anterior y siguiente permiten la implementación de la lista doblemente enlazada, facilitando la navegación bidireccional y las operaciones de inserción y eliminación eficientes.

Torre

La estructura Torre representa cada torre defensiva construida por el jugador. Cada torre tiene un id único que la identifica en el sistema. El nombre es asignado por el jugador para personalizar



sus torres. El tipo define la categoría de la torre, que puede ser "arquero", "canon" o "mago", cada una con características predefinidas. La posicion indica dónde está ubicada la torre en el camino lineal, y desde allí atacará a los enemigos que estén dentro de su rango. El danio es la cantidad de vida que resta a los enemigos cuando la torre impacta. El rango determina la distancia máxima a la que puede atacar, medida en posiciones. El costo es el precio que debe pagar el jugador para construir la torre. El campo activa indica si la torre está operativa, permitiendo desactivar torres temporalmente si se desea.

NodoOleada

La estructura NodoOleada representa un grupo de enemigos que aparecerán juntos en una oleada. El idOleada es un identificador único que permite referenciar cada oleada registrada. La cantidadEnemigos indica cuántos enemigos individuales compondrán esta oleada cuando sea invocada. El tipoEnemigo define qué clase de enemigos aparecerán, pudiendo ser "normal", "rapido" o "fuerte". La vidaBase es la cantidad de salud con la que comenzarán todos los enemigos de esta oleada. La velocidadBase determina cuántas posiciones avanzarán por turno. La recompensaBase es el dinero que se otorgará por cada enemigo de esta oleada que sea destruido. La posicionInicial está siempre establecida en 0, asegurando que todos los enemigos comienzan desde el inicio del camino. El puntero siguiente permite la implementación de la lista circular, que organiza las oleadas en orden secuencial.

Clases Principales

ListaDoblementeLazada

Esta clase implementa una lista doblemente enlazada para gestionar los enemigos activos en el campo de batalla. Utiliza punteros anterior y siguiente en cada nodo para permitir la navegación en ambas direcciones, lo que facilita operaciones como mostrar los enemigos en orden inverso. La clase mantiene punteros al primer y último nodo, así como un contador de tamaño que permite conocer rápidamente cuántos enemigos están presentes.

El constructor inicializa la lista vacía estableciendo primero, ultimo y tamaño en valores nulos o cero. El destructor recorre toda la lista liberando la memoria de cada nodo para evitar fugas de memoria.

El método insertarAlFinal crea un nuevo nodo con los datos proporcionados y lo coloca al final de la lista. Si la lista está vacía, el nuevo nodo se convierte tanto en primero como en último. En caso contrario, se enlaza con el nodo actualmente último y se actualiza el puntero ultimo.

El método eliminarPorId busca un enemigo por su identificador y lo elimina de la lista. Maneja correctamente los casos especiales donde el nodo a eliminar es el primero o el último, actualizando los punteros correspondientes. También ajusta los punteros anterior y siguiente de los nodos vecinos para mantener la integridad de la lista.

El método buscarPorId recorre la lista desde el primer nodo hasta encontrar el enemigo con el ID solicitado. Si no se encuentra, retorna nullptr.

El método actualizarPosiciones recorre todos los enemigos y suma su velocidad a su posición actual, simulando su avance en el campo de batalla.

El método aplicarDanio busca un enemigo por su ID y resta la cantidad de daño especificada de su vida, reflejando el impacto de los ataques de las torres.

ListaSecuencial

Esta clase implementa un arreglo dinámico para gestionar las torres construidas por el jugador. A diferencia de las listas enlazadas, esta estructura utiliza memoria contigua, lo que permite acceso aleatorio eficiente por índice. La capacidad inicial es de 10 elementos, pero se redimensiona automáticamente cuando se alcanza el límite, duplicando su tamaño para acomodar más torres.



El constructor reserva memoria para el arreglo de torres con la capacidad inicial especificada. El destructor libera la memoria asignada para evitar fugas.

El método insertar agrega una nueva torre al final del arreglo. Si el arreglo está lleno, redimensiona creando un nuevo arreglo del doble de tamaño, copiando todas las torres existentes y luego liberando el arreglo original.

El método eliminar busca una torre por su ID y la elimina desplazando todos los elementos posteriores una posición hacia la izquierda, manteniendo el orden de las torres restantes.

El método buscar recorre el arreglo linealmente hasta encontrar la torre con el ID solicitado, retornando un puntero a ella o nullptr si no existe.

El método obtener retorna un puntero a la torre en el índice especificado, permitiendo acceso aleatorio a las torres por su posición en el arreglo.

ListaCircular

Esta clase implementa una lista circular para gestionar las oleadas de enemigos. A diferencia de las listas lineales, el último nodo apunta al primero, formando un ciclo. Esto es útil para representar la secuencia de oleadas que se repetirán en orden. La clase mantiene un puntero al último nodo y un puntero actual que indica la próxima oleada a ser invocada.

El destructor maneja cuidadosamente la liberación de memoria en una estructura circular. Primero obtiene una referencia al primer nodo a través de ultimo->siguiente, luego recorre todos los nodos eliminándolos hasta regresar al punto de inicio.

El método registrarOleada crea un nuevo nodo con los datos proporcionados. La posición inicial siempre se establece en 0, cumpliendo con la regla de que todos los enemigos comienzan desde el inicio del camino. Si la lista está vacía, el nuevo nodo se apunta a sí mismo. En caso contrario, se inserta después del último nodo y antes del primero.

El método avanzarOleada mueve el puntero actual al siguiente nodo en la lista circular, preparando la próxima oleada para ser invocada cuando el jugador decida iniciarla.

El método obtenerActual retorna el puntero a la oleada actual, que será la siguiente en ser invocada cuando el jugador inicie una nueva oleada.

Métodos de la Clase Juego

Gestión de Torres

El método registrarTorre permite al jugador construir una torre con valores predefinidos según su tipo. Al recibir el nombre, tipo y posición, el método convierte el tipo a minúsculas para comparación y asigna los valores correspondientes de daño, rango y costo. Para el tipo "arquero" se asignan 25 de daño, rango 3 y costo 100. Para "canon" se asignan 35 de daño, rango 4 y costo 170. Para "mago" se asignan 40 de daño, rango 5 y costo 200. Antes de crear la torre, verifica que el jugador tenga suficiente dinero; si no es así, retorna false. Si el dinero es suficiente, resta el costo del dinero disponible y almacena la torre en la lista secuencial.

El método registrarTorrePersonalizada ofrece flexibilidad al jugador permitiéndole definir todos los atributos de la torre manualmente. Al igual que el método anterior, verifica el saldo disponible antes de realizar la construcción. El jugador puede especificar el nombre, tipo, posición, daño, rango y costo de la torre, personalizando completamente su estrategia defensiva.

El método eliminarTorre permite al jugador deshacerse de una torre existente. Busca la torre por su ID y, si la encuentra, devuelve la mitad de su costo al jugador antes de eliminarla de la lista secuencial. Esta mecánica incentiva la experimentación con diferentes configuraciones sin penalizar completamente los cambios de estrategia.

El método buscarTorre realiza una búsqueda lineal por ID en la lista de torres, retornando un puntero a la torre encontrada o nullptr si no existe.

Gestión de Oleadas



El método registrarOleadaPredefinida crea una nueva oleada con valores predefinidos según el tipo de enemigo seleccionado. Para enemigos "normal" se crean 5 enemigos con 20 de vida, velocidad 2 y recompensa 60. Para "rapido" se crean 3 enemigos con 15 de vida, velocidad 3 y recompensa 80. Para "fuerte" se crean 2 enemigos con 40 de vida, velocidad 1 y recompensa 120. Estas velocidades reducidas mejoran la jugabilidad, evitando que los enemigos se muevan demasiado rápido y permitiendo que las torres tengan oportunidades de ataque.

El método registrarOleadaPersonalizada permite al jugador definir todos los parámetros de una oleada manualmente, incluyendo cantidad de enemigos, tipo, vida, velocidad y recompensa. Se aplican restricciones a la velocidad para mantener la jugabilidad, limitándola entre 1 y 5. Las velocidades excesivas podrían hacer que los enemigos atravesaran el rango de las torres demasiado rápido.

El método buscarOleada recorre la lista circular de oleadas para encontrar aquella con el ID especificado. Dado que la lista es circular, el recorrido continúa hasta regresar al punto de inicio si no se encuentra la oleada.

Gestión del Combate

El método iniciarCombate prepara el juego para el enfrentamiento activo. Verifica que no haya un combate ya en curso y que exista al menos una oleada registrada. Si ambas condiciones se cumplen, activa la bandera combateActivo, permitiendo que los turnos comiencen a procesarse.

El método iniciarSiguienteOleada invoca la próxima oleada registrada. Obtiene la oleada actual a través del puntero actual de la lista circular y crea todos los enemigos que la componen, insertándolos en la lista doblemente enlazada de enemigos activos. Cada enemigo se crea con los atributos definidos en la oleada y con posición inicial 0. Una vez creados todos, avanza el puntero de oleada actual para preparar la siguiente.

El método aplicarDanioVida maneja el daño recibido por el jugador cuando los enemigos alcanzan la posición final. Cada vida del jugador tiene 100 puntos de salud. Cuando el daño recibido supera la vida actual, se pierde una vida completa y se restablece la salud a 100 puntos para la siguiente vida. Si todas las vidas se agotan, el juego termina.

El método avanzarTurno implementa la lógica central del combate. El orden de las acciones es crucial: primero atacan las torres, luego se verifican las muertes, después se mueven los enemigos y finalmente se verifica si algún enemigo alcanzó el final. Este orden evita que los enemigos rápidos puedan saltarse el rango de las torres moviéndose antes de recibir daño.

Durante la fase de ataque, cada torre revisa a todos los enemigos activos. Si la distancia entre la torre y el enemigo es menor o igual al rango de la torre, se aplica el daño correspondiente. Después de todos los ataques, se eliminan los enemigos cuya vida llegó a cero, otorgando su recompensa al jugador. Luego todos los enemigos avanzan según su velocidad, y finalmente los que superaron la posición 20 se eliminan causando daño al jugador.

Métodos de Estado

La clase Juego proporciona múltiples métodos para consultar el estado actual del juego. obtenerVidas retorna el número de vidas restantes del jugador. obtenerVidaActual indica la salud de la vida actual, que puede variar entre 0 y 100. obtenerTurno muestra cuántos turnos se han procesado desde el inicio del combate. obtenerEnemigoDestruidos lleva la cuenta de enemigos eliminados exitosamente. obtenerTorresCount, obtenerEnemigosCount y obtenerOleadasCount retornan la cantidad de elementos en cada estructura. obtenerDinero devuelve el saldo actual del jugador. enCombate indica si el juego se encuentra en modo combate activo. estaActivo verifica que el juego continúe, considerando que el jugador tiene vidas restantes.

Funciones de Interfaz de Usuario

Funciones de Visualización



La función mostrarIntro presenta al jugador la pantalla de bienvenida con información relevante como el dinero inicial, el sistema de vidas con 100 HP cada una, y las recompensas por tipo de enemigo. Utiliza caracteres ASCII básicos para formar bordes y tablas, manteniendo compatibilidad con cualquier terminal.

La función mostrarMapa genera una representación visual del campo de batalla. Crea un arreglo de 21 posiciones y coloca símbolos representativos: 'A' para torres arquero, 'C' para cañones, 'M' para magos, 'N' para enemigos normales, 'R' para rápidos, 'F' para fuertes y '.' para espacios vacíos. Esta visualización permite al jugador comprender rápidamente la distribución de sus defensas y la posición de los enemigos.

La función mostrarTorres presenta una tabla con todas las torres construidas, mostrando ID, nombre, tipo, posición, daño, rango, costo y estado. La tabla está formateada con bordes de líneas y columnas alineadas para facilitar la lectura.

La función mostrarOleadas lista todas las oleadas registradas con sus características: ID, cantidad de enemigos, tipo, vida, velocidad y recompensa. Al igual que la tabla de torres, utiliza un formato tabular claro.

La función mostrarEnemigosActivos muestra los enemigos actualmente en el campo de batalla con sus estadísticas en tiempo real. Esta información es crucial para que el jugador decida estrategias de construcción de torres.

La función mostrarEstadoCombate consolida toda la información del juego en una pantalla única. Muestra el turno actual, las vidas con una barra de salud visual usando caracteres '#' para representar los puntos de vida, el dinero disponible, las estadísticas de enemigos y torres, y luego incluye el mapa visual, la lista de torres y la lista de enemigos activos.

Funciones de Menú

La función mostrarMenuPrincipal presenta las opciones principales del juego en un formato jerárquico: opción 1 para gestionar torres, opción 2 para gestionar oleadas, opción 3 para mostrar oleadas registradas, opción 4 para mostrar el estado del combate, opción 5 para iniciar el combate y opción 6 para salir.

La función menuTorres maneja el submenú de torres con opciones para registrar torres predefinidas o personalizadas, mostrar torres registradas, eliminar torres y buscar torres por ID. Cada opción llama a los métodos correspondientes de la clase Juego y maneja la entrada de usuario con validación básica.

La función menuOleadas maneja el submenú de oleadas con opciones para registrar oleadas predefinidas o personalizadas, mostrar oleadas registradas y buscar oleadas por ID.

Funciones de Procesamiento

La función procesarAvanzarTurno maneja la ejecución de un turno de combate. Verifica que el combate esté activo y que existan enemigos antes de proceder. Llama al método avanzarTurno del juego y luego muestra los resultados del turno, incluyendo si hubo ataques, si los enemigos se movieron, las vidas restantes y el dinero actual. Finalmente muestra el mapa actualizado para que el jugador pueda visualizar los cambios.

La función limpiarBuffer se utiliza para limpiar el flujo de entrada después de operaciones que pueden dejar caracteres residuales, previniendo errores en la lectura de datos.

Flujo del Juego

Fase de Preparación

Al iniciar el juego, el jugador se encuentra en la fase de preparación donde puede construir torres y registrar oleadas. Cada torre construida cuesta dinero de su saldo inicial de \$400, incentivando decisiones estratégicas sobre qué tipos de torres colocar y en qué posiciones. Las torres pueden ubicarse en cualquier posición entre 0 y 20, y su efectividad depende de su rango de ataque y



daño. Las oleadas se registran en la lista circular, preparando la secuencia de enfrentamientos que vendrán.

Fase de Combate

Cuando el jugador inicia el combate, se activa la bandera de combate y se permite invocar oleadas. La primera oleada se invoca manualmente, creando los enemigos correspondientes en la posición 0. A partir de entonces, el jugador puede avanzar turnos, durante los cuales ocurre automáticamente el ciclo de ataque-movimiento-verificación. Cada turno puede resultar en enemigos destruidos que otorgan dinero para nuevas torres, o enemigos que alcanzan el final reduciendo la salud del jugador.

Condiciones de Finalización

El juego finaliza bajo dos condiciones. La primera es cuando el jugador pierde todas sus vidas, momento en el cual se muestra la pantalla de Game Over con estadísticas finales. La segunda es cuando se completan todas las oleadas registradas sin perder todas las vidas, momento en el cual el jugador es felicitado por su victoria.

Flujo de Datos

Los datos fluyen de manera organizada entre las estructuras. La lista secuencial almacena las torres con acceso por índice para las operaciones de ataque. La lista doblemente enlazada mantiene los enemigos activos, permitiendo eliminaciones eficientes cuando son destruidos o alcanzan el final. La lista circular organiza las oleadas, permitiendo iterar secuencialmente sobre ellas. La clase Juego coordina todas estas estructuras, asegurando que las operaciones se realicen en el orden correcto y que el estado del juego se mantenga consistente.

2.7 Resultados obtenidos

Se logró implementar completamente una lista doblemente enlazada funcional que gestiona los enemigos activos en el campo de batalla. Esta estructura permite insertar nuevos enemigos al final de la lista cuando comienza una oleada, eliminar enemigos por su identificador cuando son destruidos o alcanzan la base, buscar enemigos específicos para aplicarles daño, y recorrer la lista en ambas direcciones, lo que facilita la visualización de enemigos tanto en orden normal como inverso. La implementación maneja correctamente todos los casos especiales como la inserción en lista vacía, eliminación del primer o último elemento, y la correcta actualización de punteros anterior y siguiente para mantener la integridad de la estructura. Se verificó que no existen fugas de memoria gracias a la implementación adecuada del destructor que libera todos los nodos antes de finalizar el programa.

Se desarrolló exitosamente una lista secuencial dinámica que administra las torres construidas por el jugador con capacidad de redimensionamiento automático. Esta estructura comienza con capacidad para 10 torres y cuando se alcanza este límite, duplica su tamaño, copia todas las torres existentes al nuevo arreglo y libera la memoria anterior, garantizando que nunca haya restricciones artificiales en la cantidad de torres que el jugador puede construir. Las operaciones de inserción, eliminación y búsqueda funcionan correctamente, con la eliminación desplazando los elementos posteriores para mantener el orden contiguo. El acceso aleatorio por índice permite a las torres atacar eficientemente a los enemigos en cada turno sin necesidad de recorrer estructuras adicionales.

Se implementó una lista circular que organiza las oleadas de enemigos en secuencia, permitiendo registrar nuevas oleadas al final de la lista y avanzar progresivamente a través de ellas durante el combate. Esta estructura mantiene un puntero a la oleada actual, que indica cuál será la siguiente



en ser invocada cuando el jugador inicie una nueva oleada. El destructor maneja correctamente la liberación de memoria en una estructura circular, evitando referencias cíclicas que podrían causar problemas. Las oleadas registradas pueden ser predefinidas con valores equilibrados o personalizadas con parámetros definidos por el jugador, ofreciendo flexibilidad en la creación de desafíos.

El sistema de combate funciona correctamente procesando cada turno en el orden establecido. Las torres atacan primero a todos los enemigos dentro de su rango, luego se eliminan los enemigos cuya vida llega a cero otorgando sus recompensas al jugador, después todos los enemigos avanzan según su velocidad, y finalmente se eliminan aquellos que alcanzaron la posición 20 aplicando daño a la vida del jugador equivalente a su vida restante. Este orden garantiza que los enemigos no puedan escapar del rango de las torres sin recibir daño y que el jugador reciba las recompensas antes de que los enemigos tengan oportunidad de moverse nuevamente.

La interfaz de usuario desarrollada proporciona una experiencia clara y organizada. El menú jerárquico permite al jugador navegar fácilmente entre las diferentes funcionalidades, con submenús para torres y oleadas que agrupan operaciones relacionadas. La visualización del mapa muestra el campo de batalla con representaciones ASCII de torres (A, C, M) y enemigos (N, R, F) en sus posiciones actuales, permitiendo al jugador comprender rápidamente la situación táctica. Las tablas de torres, oleadas y enemigos presentan la información de manera estructurada con columnas alineadas y bordes que facilitan la lectura. El estado del combate incluye una barra de salud visual que representa gráficamente la vida actual del jugador, mejorando la experiencia de usuario.

2.8 Diagrama de clases





2.9 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☐ Responsabilidad

2.10 Conclusiones

El desarrollo de este proyecto ha permitido demostrar la importancia fundamental de comprender las estructuras de datos a nivel de implementación manual. Al construir desde cero una lista doblemente enlazada, una lista secuencial dinámica y una lista circular, se ha logrado un entendimiento profundo de cómo funcionan internamente estas estructuras, cómo se gestiona la memoria dinámica en C++ y cómo se optimizan las operaciones de inserción, eliminación y búsqueda. Esta experiencia ha evidenciado que, aunque las bibliotecas estándar ofrecen soluciones convenientes, el conocimiento de la implementación subyacente es crucial para desarrollar software eficiente y para resolver problemas complejos donde se requieren estructuras personalizadas adaptadas a necesidades específicas. La implementación manual ha permitido controlar cada aspecto del comportamiento de los datos, desde la creación de nodos hasta la liberación de memoria, garantizando que no existan fugas de memoria ni comportamientos inesperados durante la ejecución del juego.

La lógica de combate implementada, donde las torres atacan antes de que los enemigos se muevan, ha demostrado ser un aspecto crítico para la jugabilidad del juego. Este orden de acciones evita que los enemigos con mayor velocidad puedan evadir completamente el rango de las torres en un solo turno, lo que habría generado una experiencia frustrante para el jugador. Al procesar primero los ataques, luego las muertes, después el movimiento y finalmente la verificación de llegada a la base, se ha logrado un equilibrio donde las decisiones estratégicas del jugador sobre dónde colocar sus torres y qué tipos construir tienen un impacto significativo en el resultado del combate. Este diseño refleja cómo pequeños detalles en la lógica de programación pueden afectar drásticamente la experiencia de usuario y la sensación de control que tiene el jugador sobre el desarrollo del juego.

El sistema económico y de progresión implementado ha resultado fundamental para mantener el interés del jugador a lo largo de la partida. Al iniciar con \$400 de dinero, el jugador debe tomar decisiones estratégicas sobre qué torres construir y en qué posiciones ubicarlas, considerando que cada tipo de torre tiene un costo, daño y rango diferentes que las hacen más o menos adecuadas para diferentes situaciones. Las recompensas variables por tipo de enemigo (Normal:\$60, Rápido:\$80, Fuerte:\$120) incentivan al jugador a optimizar sus defensas para enfrentar diferentes amenazas, mientras que el sistema de vidas con 100 HP cada una proporciona un margen de error que permite al jugador recuperarse de pequeños fallos sin que la partida termine abruptamente. Esta combinación de elementos económicos y de progresión ha creado un ciclo de juego adictivo



donde construir torres genera dinero para construir más torres, permitiendo al jugador sentir un avance constante.

La arquitectura modular implementada, con archivos separados para cada estructura de datos y una clase Juego que coordina todas las funcionalidades, ha demostrado ser altamente efectiva para mantener el código organizado, legible y mantenible. Cada clase tiene una responsabilidad claramente definida: las estructuras de datos gestionan colecciones específicas de elementos, la clase Juego orquesta la lógica del juego, y el archivo main maneja exclusivamente la interfaz de usuario y el flujo del menú. Esta separación de responsabilidades facilita la identificación y corrección de errores, permite realizar modificaciones en una parte del código sin afectar otras, y hace posible que diferentes programadores trabajen en componentes distintos sin conflictos. Además, el uso de ASCII básico para la interfaz de usuario garantiza compatibilidad con cualquier terminal y simplifica la implementación visual sin necesidad de bibliotecas gráficas complejas.

2.11 Recomendaciones

Se recomienda implementar un sistema de guardado y carga de partidas que permita al jugador retomar su progreso en otro momento. Actualmente, el juego comienza desde cero cada vez que se ejecuta, lo que limita la experiencia para partidas más largas. Un sistema de persistencia que guarde en un archivo el estado de las torres, las oleadas registradas, el dinero, las vidas y el turno actual permitiría al jugador invertir tiempo en estrategias más elaboradas sin temor a perder su progreso. Este sistema podría implementarse utilizando la biblioteca fstream para escribir y leer datos estructurados, serializando las estructuras de datos de manera que puedan reconstruirse al cargar una partida guardada.

Se sugiere agregar más variedad de torres y enemigos para aumentar la profundidad estratégica del juego. Actualmente existen tres tipos de torres y tres tipos de enemigos, lo que proporciona una base sólida pero limitada. Se podrían incorporar torres con efectos especiales como ralentización de enemigos, daño en área que afecte a múltiples enemigos en una misma posición, o torres que generen dinero adicional por cada enemigo destruido. De manera similar, se podrían agregar enemigos con habilidades especiales como resistencia a cierto tipo de daño, capacidad de curarse a sí mismos, o que dejen bonificaciones al ser destruidos. Esta variedad aumentaría significativamente la rejugabilidad y permitiría al jugador desarrollar estrategias más complejas.

Se recomienda implementar un sistema de dificultad progresiva donde las oleadas no solo tengan más enemigos, sino también enemigos con estadísticas mejoradas que escalen con el número de oleada completada. Actualmente las oleadas predefinidas tienen valores fijos, lo que puede hacer que el juego se vuelva predecible después de varias partidas. Un sistema de escalado donde la vida, velocidad y recompensa de los enemigos aumenten gradualmente según el número de oleada mantendría el desafío constante a lo largo del juego. Además, se podría implementar un modo "sin fin" donde después de completar todas las oleadas registradas, el juego continúe generando oleadas cada vez más difíciles hasta que el jugador finalmente sea derrotado.

Se sugiere mejorar la interfaz de usuario con colores utilizando códigos ANSI para terminales que lo soporten. Aunque el juego actualmente utiliza ASCII básico para garantizar compatibilidad universal, la incorporación de colores podría mejorar significativamente la experiencia visual sin comprometer la portabilidad en terminales modernas. Por ejemplo, se podrían mostrar las torres en color azul, los enemigos en rojo, las vidas en verde y los mensajes de advertencia en amarillo. Esto ayudaría al jugador a identificar rápidamente información crítica sin necesidad de leer



detenidamente cada línea. Es importante que esta funcionalidad sea opcional o que incluya una detección automática de soporte para colores para mantener compatibilidad con terminales más básicas.

Se recomienda agregar estadísticas más detalladas y un sistema de logros para motivar al jugador a experimentar con diferentes estrategias. Actualmente el juego muestra estadísticas básicas como turnos jugados y enemigos destruidos, pero se podrían añadir métricas más específicas como torres construidas por tipo, daño total infligido, dinero total gastado, eficiencia por torre, entre otras. Los logros podrían incluir metas como "construir una torre de cada tipo", "destruir 100 enemigos", "completar una partida sin perder vidas" o "alcanzar el turno 50", proporcionando objetivos adicionales que extiendan la vida útil del juego y motiven al jugador a explorar todas las mecánicas implementadas.

2.12 Anexos

<https://github.com/DiegO2255/Prueba-Practica-Grupo-7>

Ilustración 1. Enemigo.h

```
Enemigo.h  Torre.h  Juego.h  main.cpp  Oleada.h

1  #ifndef ENEMIGO_H
2  #define ENEMIGO_H
3
4  #include <string>
5
6  using namespace std;
7
8  struct NodoEnemigo {
9      int id;
10     string tipo;
11     int vida;
12     int velocidad;
13     int posicion;
14     int recompensa;
15
16     NodoEnemigo* anterior;
17     NodoEnemigo* siguiente;
18
19     NodoEnemigo(int _id, string _tipo, int _vida, int _vel, int _pos, int _rec)
20         : id(_id), tipo(_tipo), vida(_vida), velocidad(_vel), posicion(_pos), recompensa(_rec),
21         anterior(nullptr), siguiente(nullptr) {}
22 };
23
24 class ListaDoblementeLazada {
25 private:
26     NodoEnemigo* primero;
27     NodoEnemigo* ultimo;
28     int tamaño;
29
30 public:
31     ListaDoblementeLazada() : primero(nullptr), ultimo(nullptr), tamaño(0) {}
32
33     ~ListaDoblementeLazada() {
34         NodoEnemigo* actual = primero;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Ilustración 2. Torre.h

```
Enemigo.h  Torre.h  Juego.h  main.cpp  Oleada.h
1  #ifndef TORRE_H
2  #define TORRE_H
3
4  #include <string>
5
6  using namespace std;
7
8  struct Torre {
9      int id;
10     string nombre;
11     string tipo;
12     int posicion;
13     int danio;
14     int rango;
15     int costo;
16     bool activa;
17
18     Torre() : id(0), nombre(""), tipo(""), posicion(0), danio(0), rango(0), costo(0), activa(true) {}
19
20     Torre(int _id, string _nombre, string _tipo, int _pos, int _dan, int _ran, int _cos)
21         : id(_id), nombre(_nombre), tipo(_tipo), posicion(_pos), danio(_dan), rango(_ran), costo(_cos), activa(true) {}
22 };
23
24 class ListaSecuencial {
25 private:
26     Torre* torres;
27     int capacidad;
28     int tamaño;
29
30 public:
31     ListaSecuencial(int cap = 10) : capacidad(cap), tamaño(0) {
32         torres = new Torre[capacidad];
33     }
34 }
```

Ilustración 3. Juego.h

```
Enemigo.h  Torre.h  Juego.h  main.cpp  Oleada.h
1  #ifndef JUEGO_H
2  #define JUEGO_H
3
4  #include "Torre.h"
5  #include "Enemigo.h"
6  #include "Oleada.h"
7  #include <iostream>
8  using namespace std;
9
10 class Juego {
11 private:
12     ListaSecuencial torres;
13     ListaDoblementeEnlazada enemigos;
14     ListaCircular oleadas;
15
16     int vidasJugador;
17     int vidaActual;
18     int enemigoDestruidos;
19     int generadorIdEnemigo;
20     int generadorIdTorre;
21     int generadorIdOleada;
22     int turno;
23     bool juegoActivo;
24     int dinero;
25     bool combateActivo;
26
27 public:
28     Juego() : vidasJugador(3), vidaActual(100), enemigoDestruidos(0), generadorIdEnemigo(1),
29             generadorIdTorre(1), generadorIdOleada(1), turno(0), juegoActivo(true),
30             dinero(400), combateActivo(false) {}
31
32     // ===== METODOS PARA TORRES =====
33
34     bool registrarTorre(string nombre, string tipo, int posicion) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Ilustración 4. Oleada.h

```
Enemigo.h Torre.h Juego.h main.cpp Oleada.h
1  #ifndef OLEADA_H
2  #define OLEADA_H
3
4  #include <string>
5
6  using namespace std;
7
8  struct NodoOleada {
9      int idOleada;
10     int cantidadEnemigos;
11     string tipoEnemigo;
12     int vidaBase;
13     int velocidadBase;
14     int recompensaBase;
15     int posicionInicial;
16
17     NodoOleada* siguiente;
18
19     NodoOleada(int _id, int _cant, string _tipo, int _vida, int _vel, int _rec, int _pos)
20         : idOleada(_id), cantidadEnemigos(_cant), tipoEnemigo(_tipo), vidaBase(_vida), velocidadBase(_vel),
21           recompensaBase(_rec), posicionInicial(_pos), siguiente(nullptr) {}
22 };
23
24 class ListaCircular {
25 private:
26     NodoOleada* ultimo;
27     NodoOleada* actual;
28     int tamano;
29
30 public:
31     ListaCircular() : ultimo(nullptr), actual(nullptr), tamano(0) {}
32
33     ~ListaCircular() {
34         if (ultimo == nullptr) return;
```

Ilustración 5. Main

```
Enemigo.h Torre.h Juego.h main.cpp Oleada.h
10
11 // Prototipos de funciones
12 void mostrarMenuPrincipal();
13 void menuTorres(Juego& juego);
14 void menuOleadas(Juego& juego);
15 void mostrarOleadas(Juego& juego);
16 void mostrarEstadoCombate(Juego& juego);
17 void mostrarMapa(Juego& juego);
18 void mostrarEnemigosActivos(Juego& juego);
19 void procesarAvanzarTurno(Juego& juego);
20 void mostrarIntro();
21 void mostrarGameOver(Juego& juego);
22 void limpiarBuffer();
23
24 // ===== FUNCIONES DE VISUALIZACION =====
25
26 void mostrarIntro() {
27     cout << "\n+-----+<< endl;
28     cout << "          BIENVENIDO A TOWER DEFENSE          |<< endl;
29     cout << "+-----+<< endl;
30     cout << "| Defiende tu base de los enemigos!           |<< endl;
31     cout << "|                                           |<< endl;
32     cout << "| Dinero inicial: $400                        |<< endl;
33     cout << "| Vidas: 3 (cada vida tiene 100 HP)           |<< endl;
34     cout << "|                                           |<< endl;
35     cout << "| RECOMPENSAS POR ENEMIGO:                   |<< endl;
36     cout << "| * Normal: $60 (Vida:20, Velocidad:2)        |<< endl;
37     cout << "| * Rapido: $80 (Vida:15, Velocidad:3)        |<< endl;
38     cout << "| * Fuerte: $120 (Vida:40, Velocidad:1)       |<< endl;
39     cout << "+-----+<< endl;
40 }
41
42 void mostrarGameOver(Juego& juego) {
43     cout << "\n+-----+<< endl;
```

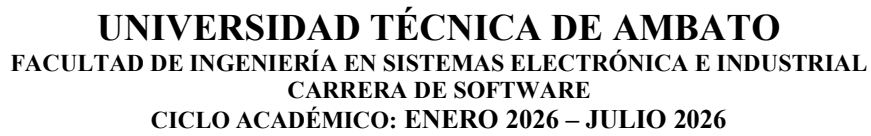



Ilustración 6. Compilador

```
C:\Users\Joss\Desktop\prueb: X + v
```

```
+=====+  
|          BIENVENIDO A TOWER DEFENSE          |  
+-----+  
| Defiende tu base de los enemigos!             |  
  
| Dinero inicial: $400                          |  
| Vidas: 3 (cada vida tiene 100 HP)              |  
  
| RECOMPENSAS POR ENEMIGO:                      |  
| * Normal: $60 (Vida:20, Velocidad:2)           |  
| * Rapido: $80 (Vida:15, Velocidad:3)           |  
| * Fuerte: $120 (Vida:40, Velocidad:1)         |  
+=====+
```

```
+=====+  
|          MENU PRINCIPAL                        |  
+-----+  
| 1. Torres                                     |  
| 2. Oleadas (Enemigos)                       |  
| 3. Mostrar oleadas registradas               |  
| 4. Mostrar estado del combate                 |  
| 5. Iniciar combate                           |  
| 6. Salir                                      |  
+=====+
```

```
Seleccione una opcion:
```